



BERKELEY LAB

Bringing Science Solutions to the World



U.S. DEPARTMENT OF
ENERGY

Office of Science

Just Write Fortran:

Experiences with a Language-Based Alternative to MPI+X

Damian Rouson, Baboucarr Dibba, Katherine Rasmussen, Brad Richardson, David Torres, Yunhao Zhang
Berkeley Lab

Ethan Gutmann
NCAR

Kareem Ergawy, Michael Klemm
Advanced Micro Devices, Inc.

Sameer Shende
University of Oregon

Parallel Applications Workshop — Alternatives to MPI+X (PAW-ATM), 17 November 2024



Overview

Just Write Fortran:

01

Motivation

02

Background:
Parallelism in
Fortran 2023

03

User Experience:
Fun with Compilers

04

Discussion of Results

05

Conclusions and Future Work



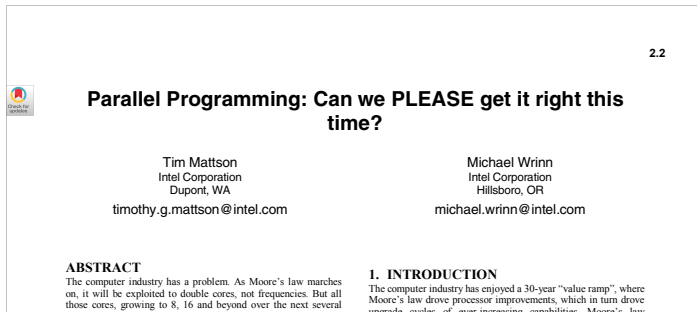
BERKELEY LAB

Bringing Science Solutions to the World

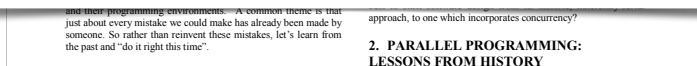
Background



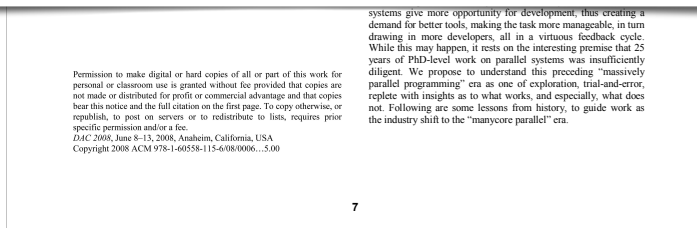
U.S. DEPARTMENT OF
ENERGY
Office of Science



"With few exceptions, only graduate students and other strange people write parallel software."



"An ideal solution would automatically exploit concurrency through techniques such as...automatic parallelization of loops."



T. Mattson and M. Wrinn (2008)
Proceedings of the 45th annual design automation conference, pp. 7–11.

Overview

Just Write Fortran:

01

Motivation

02

Background:
Parallelism in
Fortran 2023

03

User Experience:
Fun with Compilers

04

Discussion of Results

05

Conclusions and Future Work

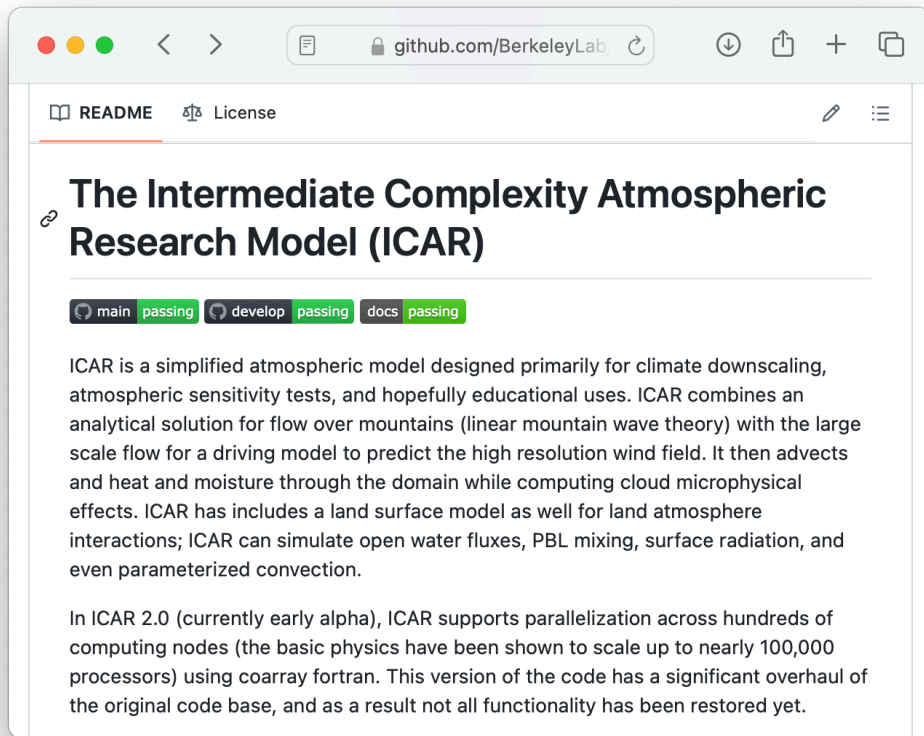
Fortran 2023 Parallelism

Multi-Image Execution (“Coarray Fortran”):

-  Coarrays
-  Synchronization
-  Events
-  Notifications
-  Locks
-  Failed images
-  Teams
-  Critical sections
-  Intrinsic functions
-  Collective subroutines
-  Atomic subroutines
-  Types
-  Values
-  Atomic kind type parameters

Gutmann, E. D., I. Barstad, M. P. Clark, J. R. Arnold, and R. M. Rasmussen (2016), *The Intermediate Complexity Atmospheric Research Model*, J. Hydrometeor., doi:[10.1175/JHM-D-15-0155.1](https://doi.org/10.1175/JHM-D-15-0155.1).

Application:



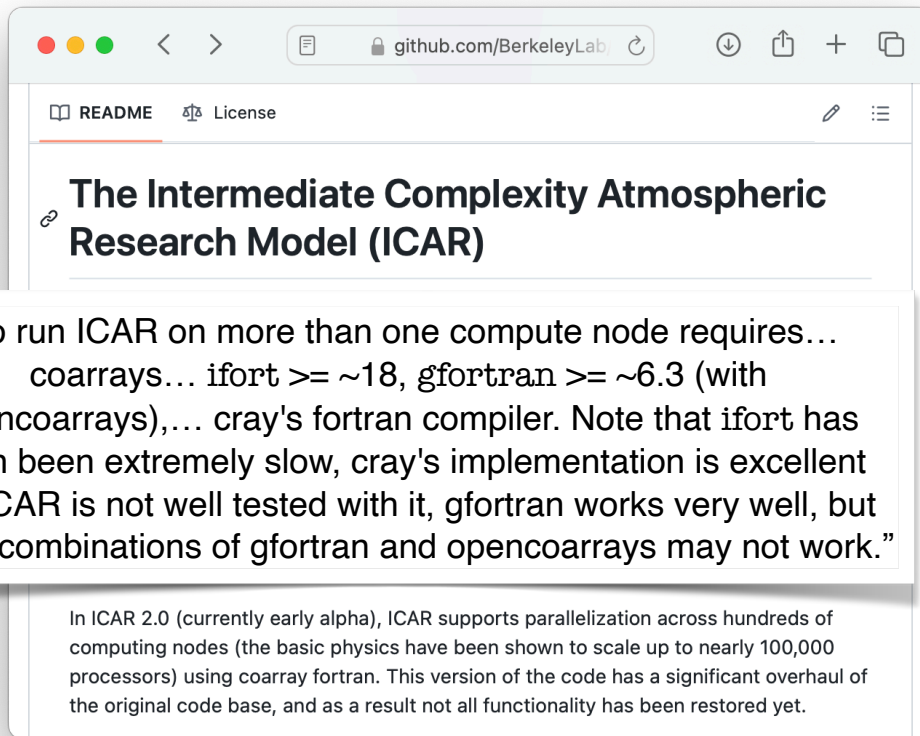
Fortran 2023 Parallelism

Multi-Image Execution (“Coarray Fortran”):

- ☕ Coarrays
- ☕ Synchronization
- ☕ Events
- ☕ Notifications
- ☕ Locks
- ☕ Failed images
- ☕ Teams
- ☕ Critical sections
- ☕ Intrinsic functions
- ☕ Collective subroutines
- ☕ Atomic subroutines
- ☕ Types
- ☕ Values
- ☕ Atomic kind type parameters

Gutmann, E. D., I. Barstad, M. P. Clark, J. R. Arnold, and R. M. Rasmussen (2016), *The Intermediate Complexity Atmospheric Research Model*, J. Hydrometeor., doi:[10.1175/JHM-D-15-0155.1](https://doi.org/10.1175/JHM-D-15-0155.1).

Application:



“To run ICAR on more than one compute node requires... coarrays... ifort $\geq \sim 18$, gfortran $\geq \sim 6.3$ (with opencoarrays),... cray's fortran compiler. Note that ifort has often been extremely slow, cray's implementation is excellent but ICAR is not well tested with it, gfortran works very well, but some combinations of gfortran and opencoarrays may not work.”

In ICAR 2.0 (currently early alpha), ICAR supports parallelization across hundreds of computing nodes (the basic physics have been shown to scale up to nearly 100,000 processors) using coarray fortran. This version of the code has a significant overhaul of the original code base, and as a result not all functionality has been restored yet.

Fortran 2023 Parallelism

Statement-/Construct Parallelism:

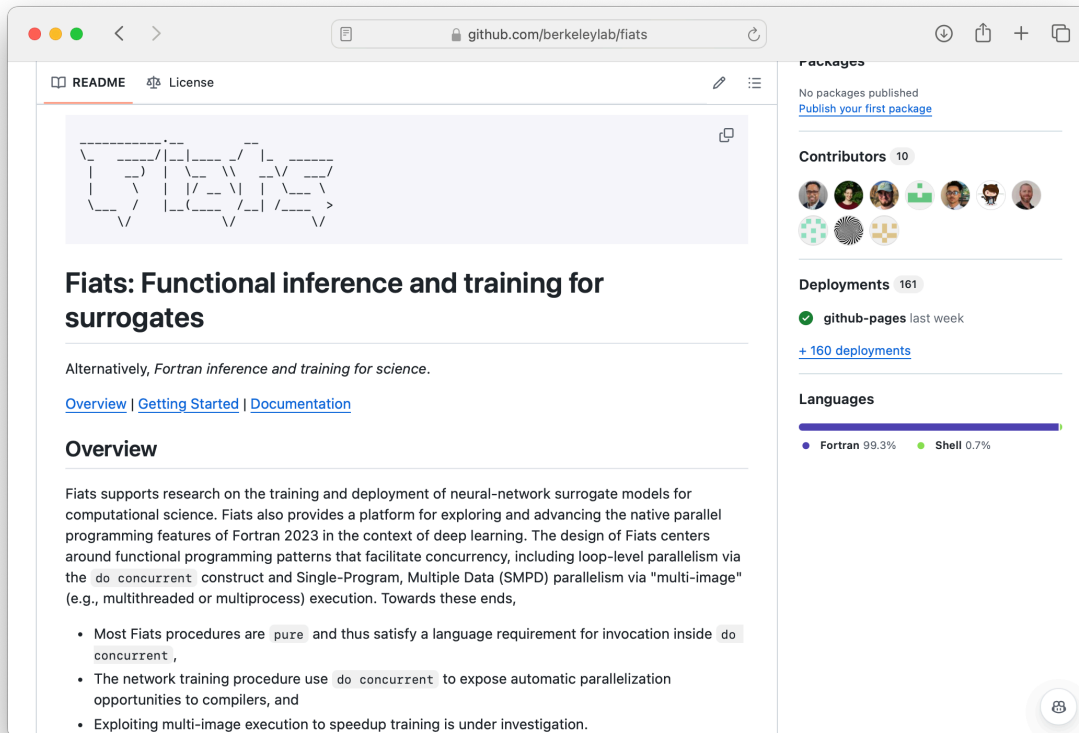
 `do concurrent`

- pure procedures

 Array statements

- elemental procedures
- Intrinsic functions:
`matmul`, `pack`, ...

Library:



Fortran 2023 Parallelism



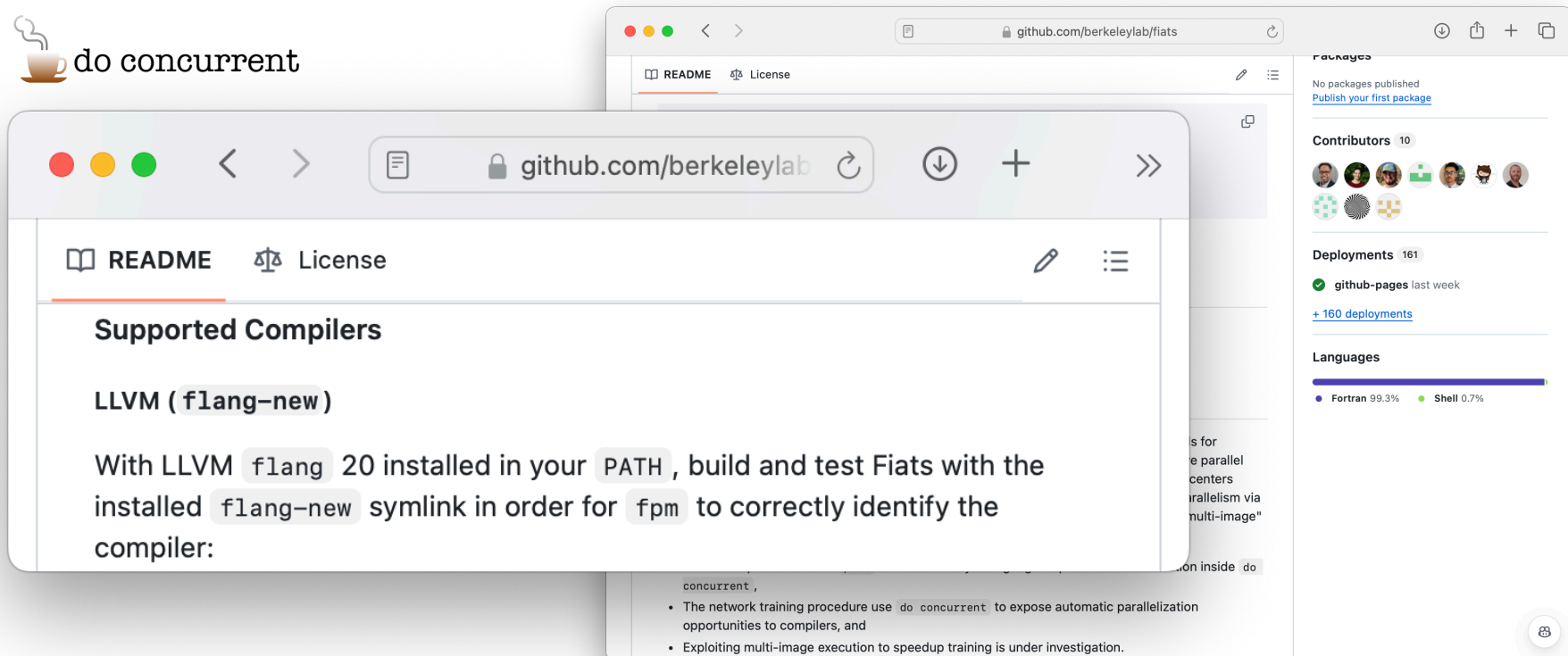
BERKELEY LAB

Bringing Science Solutions to the World

Statement-/Construct Parallelism:

 `do concurrent`

Library:



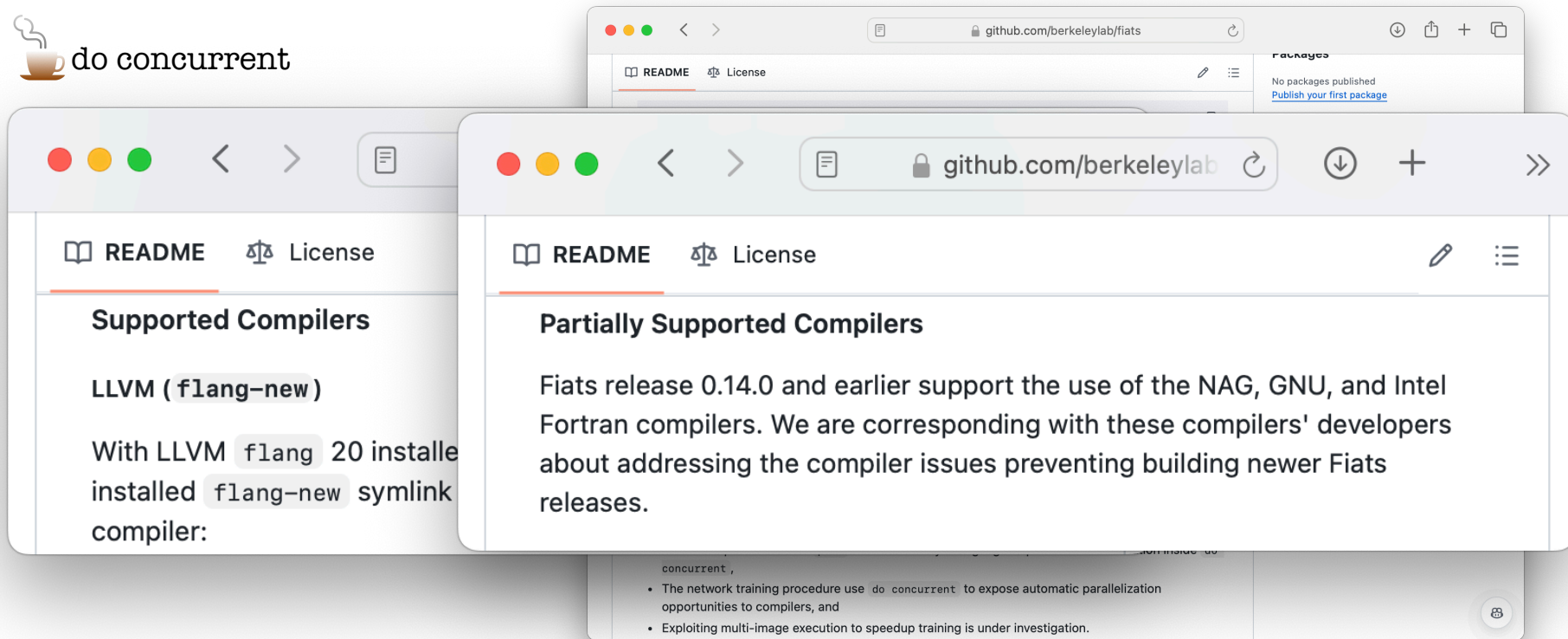
The screenshot displays the GitHub repository for `berkeleylab/fiats`. The main content is the README, which is titled "Supported Compilers". It specifies that LLVM (flang-new) is required. The text states: "With LLVM flang 20 installed in your PATH, build and test Fiats with the installed flang-new symlink in order for fpm to correctly identify the compiler:". Below this, there is a list of bullet points: "concurrent," "The network training procedure use do concurrent to expose automatic parallelization opportunities to compilers, and", and "Exploiting multi-image execution to speedup training is under investigation." The right sidebar shows package statistics: "No packages published", "Contributors 10", "Deployments 161" (with a link to "github-pages last week" and "+ 160 deployments"), and "Languages" (Fortran 99.3%, Shell 0.7%).

Fortran 2023 Parallelism

Statement-/Construct Parallelism:

 `do concurrent`

Library:



The image shows three overlapping browser windows displaying the GitHub repository for Berkeley Lab's Fiats Fortran library. The top window shows the repository overview with tabs for README and License. The middle window shows the README content, specifically the 'Supported Compilers' section, which lists LLVM (flang-new) and mentions that with LLVM flang 20 installed, flang-new symlink compiler is available. The bottom window shows the 'Partially Supported Compilers' section, stating that Fiats release 0.14.0 and earlier support the use of the NAG, GNU, and Intel Fortran compilers, and that they are corresponding with these compilers' developers about addressing the compiler issues preventing building newer Fiats releases. A snippet of code is visible at the bottom of the bottom window, showing `do concurrent` and a list of bullet points.

Supported Compilers

LLVM (flang-new)

With LLVM flang 20 installed
installed flang-new symlink
compiler:

Partially Supported Compilers

Fiats release 0.14.0 and earlier support the use of the NAG, GNU, and Intel Fortran compilers. We are corresponding with these compilers' developers about addressing the compiler issues preventing building newer Fiats releases.

concurrent ,

- The network training procedure use `do concurrent` to expose automatic parallelization opportunities to compilers, and
- Exploiting multi-image execution to speedup training is under investigation.

Overview

Just Write Fortran:

01

Motivation

02

Background:
Parallelism in
Fortran 2023

03

User Experience:
Fun with Compilers

04

Discussion of Results

05

Conclusions and Future Work

Compiler Status

Multi-Image Execution:



Cray



Intel



GNU + OpenCoarrays



NAG



LLVM Flang:

- **Complete:** Parses parallel syntax
- **Recently launched:** Lowering to PRIF calls
- **In review:** PRIF 0.4 Design Document
- **Under development:** Caffeine parallel runtime library

Automatic Parallelization of do concurrent:



NVIDIA: CPU, GPU



Intel: CPU, GPU



Cray: CPU, GPU



LFortran: CPU



LLVM Flang: CPU (GPU under development)

The World's Shortest Bug Reproducer

end

Overview

Just Write Fortran:

01

Motivation

02

Background:
Parallelism in
Fortran 2023

03

User Experience:
Fun with Compilers

04

Discussion of Results

05

Conclusions and Future Work

Fiats: Inference

example — vim concurrent-inferences.f90 — 65x5

```
50 do concurrent(i=1:lat, k=1:lev, j=1:lon)
51   outputs(i,k,j) = neural_network%infer(inputs(i,k,j))
52 end do
53
```

50,1

44%

Fiats: Inference

```
example — vim concurrent-inferences.f90 — 65x5
50 do concurrent(i=1:lat, k=1:lev, j=1:lon)
51     outputs(i,k,j) = neural_network%infer(inputs(i,k,j))
52 end do
53
```

```
example — vim concurrent-inferences.f90 — 70x10
59 !$omp parallel do shared(inputs,outputs)
60     do j=1,lon
61         do k=1,lev
62             do i=1,lat
63                 outputs(i,k,j) = neural_network%infer(inputs(i,k,j))
64             end do
65         end do
66     end do
67 !$omp end parallel do

59,1 55%
```

Fiats: Inference

```
example — vim concurrent-inferences.f90 — 65x5
50 do concurrent(i=1:lat, k=1:lev, j=1:lon)
51   outputs(i,k,j) = neural_network%infer(inputs(i,k,j))
52 end do
53
```

```
example — vim concurrent-inferences.f90 — 70x10
59 !$omp parallel do shared(inputs,outputs)
60   do j=1,lon
61     do k=1,lev
62       do i=1,lat
63         outputs(i,k,j) = neural_network%infer(inputs(i,k,j))
64       end do
65     end do
66   end do
67 !$omp end parallel do
```

```
example — vim concurrent-inferences.f90 — 50x5
73 !$omp workshare
74   outputs = neural_network%infer(inputs)
75 !$omp end workshare
76
```

73,1

64%

CPU Parallelism on Perlmutter

Compiler:

Berkeley Lab [llvm-project fork](#)
git tag paw-atm24-fiats
Commits pulled from ROCm fork

Neural network:

- Activation function: GELU
- Numbers of inputs: 80
- Number of outputs: 31
- Nodes per hidden layer: 256, 384, 256

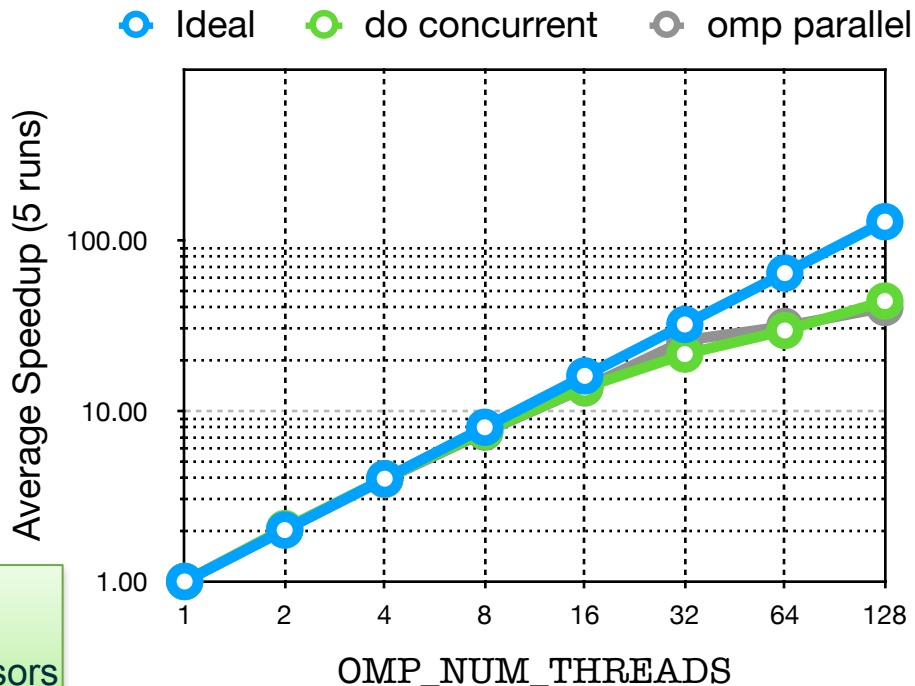
Source: Z. Bai

Platform:

Dedicated interactive node
2x 64-core AMD EPYC 7663 processors

Example Command:

```
OMP_NUM_THREADS = 128 fpm run --example concurrent-inferences \  
--runner "srun --cpu_bind=cores -c 128 -n 1" -- --network model.json
```



Fiats: Training

```
fiats — vim neural_network_s.F90 — 101x36
907 #if F2023_LOCALITY
908     iterate_through_batch: &
909     do concurrent (pair = 1:mini_batch_size) local(a,z,delta) reduce(+: dcdb, dcdw)
910
911 #elif F2018_LOCALITY
912
913     reduce_gradients: &
914     block
915         real reduce_dcdb(size(dcdb,1),size(dcdb,2),mini_batch_size)
916         real reduce_dcdw(size(dcdw,1),size(dcdw,2),size(dcdw,3),mini_batch_size)
917         reduce_dcdb = 0.
918         reduce_dcdw = 0.
919
920         iterate_through_batch: &
921         do concurrent (pair = 1:mini_batch_size) local(a,z,delta)
922
923 #else
924
925     reduce_gradients: &
926     block
927         real reduce_dcdb(size(dcdb,1),size(dcdb,2),mini_batch_size)
928         real reduce_dcdw(size(dcdw,1),size(dcdw,2),size(dcdw,3),mini_batch_size)
929         reduce_dcdb = 0.
930         reduce_dcdw = 0.
931
932         iterate_through_batch: &
933         do concurrent (pair = 1:mini_batch_size)
934
935             iteration: &
936             block
937
938                 real a(maxval(self%nodes_), input_layer:output_layer) ! Activations
939                 real z(size(b,1),size(b,2)), delta(size(b,1),size(b,2))
940 #endif
941
```

910,0-1

89%

Conclusions and Future Work

Conclusions



Fortran 2023 provides a language-based alternative to MPI+x in the form of multi-image execution + statement/construct-level parallelism.



Non-overlapping sets of compilers support either or both forms of parallelism.



Compiler implementations still vary in maturity and robustness, but the LLVM Flang CPU parallelization results are encouraging.

Future Work



Inference and training on GPUs: offloading vs embedded



Multi-image training



Ongoing development of AMD Next-Gen Fortran compiler: [blog](#).

Acknowledgements

The Berkeley Lab Fortran Team

Dan Bonachea, Hugh Kadhemi, Brad Richardson, Kate Rasmussen

Collaborators

Fiats: Zhe Bai, Jeremy Bailey, David Torres, Kareem Jabbar Weaver, Jordan Welsman, Yunhao Zhang

LLVM Flang: Jeff Hammond, Jean-Didier Paillex, Etienne Renault

OpenCoarrays: Izaak Beekman, Tobias Burnus, Alessandro Fanfarillo, Andre Vehreschild

ICAR: Ethan Gutmann

TAU: Sameer Shende